

Fly GACA — Instructor Dashboard — Design Spec

SECTION 06-operations-it

TYPE spec

STATUS **active**

OWNER Founder

UPDATED 2026-08-19

Status: partly built · **Maps to:** action plan §5.2, `roadmap.md` Phase 5 **Prepared:** 2026-05-25 · **Re-based on the Cloud Run / Postgres stack:** 2026-08-19

This is the build spec for the B2B instructor/school dashboard. It is deliberately spec-first: the hard part is not the UI, it is a multi-tenant data model that lets one user see another's progress **without** weakening the strict per-user isolation the platform has today. That has to be designed, not rushed.

The `/schools` page already *sells* this — "a school admin dashboard ... cohort exam-readiness and per-cadet progress ... is included with every contract." This spec builds that promise.

What already ships. The org-owner half of this is live: `/business/admin` renders a cohort readiness report from `GET /api/org/:orgId/cohort-readiness`, backed by the `orgs` and `org_seats` tables and the `study_progress` upload from `src/lib/services/studyProgressSync.ts`. What is **not** built is the instructor role, per-cadet drill-down, reading-path assignment, and — importantly — the consent gate in §3. Read the rest of this spec as the delta.

1. The core constraint

Today the model is strict per-user isolation, and since the Cloud Run port it is enforced *structurally* rather than declaratively: **the browser has no database access at all**. It talks only to `/api/*`, `routes/account.ts` reads and writes rows for `requireUser(req)` and nobody else, and there is simply **no route** that returns another user's profile, logbook or records.

The instructor dashboard requires **controlled cross-user reads** — an instructor seeing a cadet's progress. The entire design below exists to do that safely.

The key design decision that resolves it: the cross-user read is a **server-side join that returns a projection, never rows**. The API joins `org_seats → users → entitlements → study_progress` in one query and returns only derived readiness fields (coverage %, last mock score, ready/not-ready, `progressUpdatedAt`). No route ever hands an instructor a cadet's `profiles`, `flights` or `pilot_records` row, and the projection is computed in `org-core.ts` where it is unit-tested. The isolation guarantee is a property of the route surface, not of a rules file.

This replaces the older "each cadet's client writes a scoped copy into the school's subtree" design, which existed only because Firestore could not join server-side and its rules could not express a column-level projection. Postgres can do both, so the duplicate write — and the staleness it invited — is gone. **Reviewers: this is the single biggest mechanism change in this spec.**

2. Roles

Role	Who	Capability
Cadet / pilot	exists today — a row in <code>users</code>	Owns their data; opts in to a cohort
Org owner	exists today — <code>orgs.owner_user_id</code>	Buys seats, provisions the roster, reads the cohort rollup
Instructor	new	Reads their cohort's progress; assigns reading paths
School admin	new	Manages seats, instructors and the roster; sees the cohort rollup

A cadet on a school plan is already tagged two ways: their seat is a row in `org_seats (org_id, email)` — addressed by **email**, because seats are provisioned before the invitee has an account and claimed on their first *verified* sign-in — and their `entitlements.source` is `'school'`.

Today `orgs` has exactly one privileged actor, `owner_user_id`, and `routes/org.ts` checks it with `ownedOrg(orgId, uid)`. Instructor and admin as *distinct* roles do not exist yet; §4 adds them.

3. Consent & privacy — first-class

A cadet's logbook and study data are personal data. The PDPL DPIA is already a launch gate (`roadmap.md`); cross-user visibility **must** be inside that DPIA's scope.

Two rules this spec commits to:

1. **Scope.** Instructors see **study and exam-readiness data only** — ground-school progress, mock-exam scores, currency flags. They do **not** see the personal logbook. This one is already true by construction: `study_progress.summary` holds scores and completion only — never answers — and no route joins `flights` into the cohort report.
2. **Explicit, revocable consent.** When a cadet is enrolled into a cohort they see a consent screen naming exactly what the school will see, and must accept. They can revoke it from Settings, which flips their seat to `consent = false` and blanks the readiness fields the report returns for them.

[!WARNING] Rule 2 is not implemented. `org_seats` has no consent column, and `GET /api/org/:orgId/cohort-readiness` currently returns a readiness row for every seat on the roster. Closing this is the first item in §8 and a DPIA prerequisite — it should land before the dashboard is sold to a second school.

4. Data model (Postgres)

Nothing about the cadet's own tables changes. Two columns and one table are added; the progress table the old spec called "Phase A" **already exists**.

```
-- SHIPPED – the cadet's own progress projection, uploaded by their client
-- (studyProgressSync.ts, upload-only, best-effort). Scores + completion only.
-- CREATE TABLE study_progress (
--   user_id      uuid PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
--   summary      jsonb NOT NULL DEFAULT '{}':::jsonb,
--   updated_at   timestampz NOT NULL DEFAULT now()
-- );

-- SHIPPED – the cohort and its roster.
-- orgs          (id, name, owner_user_id, seat_limit, created_at)
-- org_seats     (org_id, email, status, source, expires_at, claimed_by, created_at)

-- NEW – consent, per seat. Written only by the cadet's own consent/revoke route.
ALTER TABLE org_seats
  ADD COLUMN consent      boolean NOT NULL DEFAULT false,
  ADD COLUMN consent_at   timestampz;

-- NEW – staff of an org, beyond the single owner.
CREATE TABLE org_members (
  org_id        uuid NOT NULL REFERENCES orgs(id) ON DELETE CASCADE,
  user_id       uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role          text NOT NULL CHECK (role IN ('admin','instructor')),
  created_at    timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (org_id, user_id)
);
CREATE INDEX org_members_user_idx ON org_members (user_id);

-- NEW – instructor → cadet reading-path assignment.
CREATE TABLE org_assignments (
  org_id        uuid NOT NULL REFERENCES orgs(id) ON DELETE CASCADE,
  email         citext NOT NULL,
  path_ids      text[] NOT NULL DEFAULT '{}',
  assigned_by   uuid REFERENCES users(id),
  updated_at    timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (org_id, email)
);
```

Denormalized cadet name/email on the roster is unnecessary — the report joins `users` on `org_seats.email` and reads `profiles` for the display name in the same query.

Seat accounting stays where it is: `checkSeatLimit` in `org-core.ts` counts `org_seats` against `orgs.seat_limit` before provisioning, so a cohort purchase cannot over-provision. `seatExpiry` caps a seat at one intake window, which is what stops a 90-day cohort from minting permanent `school` accounts.

5. Access model — how it is actually enforced

There is no rules file, no emulator, and no client-side database session. Every read is a route, and the route is the boundary. That is a straight upgrade over the Firestore design, whose whole §5 was a rules sketch that had to be red-teamed in an emulator.

The gate for every cohort route:

```
// server/src/routes/org.ts – the shipped owner check, generalised
async function orgStaff(orgId: string, uid: string): Promise<OrgRole> {
  // owner, or a row in org_members. Anything else → 403.
}
```

Rules this commits to, each testable as an ordinary integration test:

Rule	Where it lives
Only the owner or an <code>org_members</code> row may call any <code>/api/org/:orgId/*</code> route	<code>orgStaff()</code> in <code>routes/org.ts</code>
The cohort response is a projection — readiness fields only, no cadet row	<code>cohortRow()</code> in <code>org-core.ts</code>
A seat with <code>consent = false</code> returns status only, with readiness blanked	<code>cohortRow()</code> — pure, so this is a unit test
Only the cadet may set their own <code>consent</code>	a route under <code>routes/account.ts</code> , gated by <code>requireUser</code>
Only an instructor/admin of that org may write <code>org_assignments</code>	<code>routes/org.ts</code>
Seat creation/removal is server-only; there is no client write path	already true — <code>provision-seats</code> is owner-gated

We are **not** adding Postgres row-level security. RLS protects a database from a client that connects to it directly; no client connects to this database. Adding it would be ceremony that hides the real boundary, which is the route surface. The Firestore-only feature this spec loses — a declarative, independently auditable policy file — is replaced by pure `*-core.ts` policy functions plus route tests, which is the pattern the whole backend already uses.

6. Enrollment & consent flow

1. **Org created** — by Fly GACA on contract signing, via `server/scripts/grant-org.mjs` (`--dry-run` supported), not self-serve. Sets `name`, `owner_user_id`, `seat_limit`.
2. **Owner adds instructors** — by email; a new route resolves the address to a `users.id` and inserts an `org_members` row with `role = 'instructor'`.
3. **Cadets enrolled** — the owner posts cadet emails to `POST /api/org/:orgId/provision-seats` (shipped), which writes `org_seats` rows with `status = 'invited'`, `consent = false`, and a capped `expires_at`. The seat is claimed on the cadet's first **verified** sign-in — email verification is the ownership proof, the same rule that governs staff and school-domain grants.
4. **Cadet consents** — on next sign-in the cadet sees a consent screen: "Your flight school [name] will see your ground-school progress and mock-exam scores. They will not see your logbook. You can revoke this anytime in Settings." On accept, their own route sets `consent = true`, `consent_at = now()`, and the seat goes `active`.
5. **Revocation** — Settings → "Leave school cohort" sets `consent = false`. Nothing needs blanking in storage: the report projects readiness out of a non-consenting seat, so the cadet's own data stays intact and simply stops being visible.

All seat, entitlement and membership writes go through the API — exactly as `entitlements` is handled today, where **no route lets a client write its own plan**.

7. UI plan

A new instructor surface alongside the shipped `/business/admin`, both under the same `orgStaff` gate. Routes go in `src/router.tsx`, lazy-loaded, page-per-folder under `src/pages/`, with strings in **both** `en.json` and `ar.json` (the i18n parity test fails the build otherwise) and logical CSS properties so RTL mirrors.

- **Cohort view** — a table/cards: each cadet's name, exam-readiness %, last mock score, lessons complete, currency flags, last-active. Sort + filter (e.g. "show me who's behind"). This is the "see at a glance who is ready and who is falling behind" promised on `/schools`. *(Shipped for the owner; needs the instructor gate.)*
- **Per-cadet drill-down** — the cadet's readiness trend, mock-exam history, ground-school lesson map, currency. Plus an **Assign reading paths** control → writes `org_assignments`; the cadet sees them as suggested paths.
- **Admin view** — seats used / total, add/remove cadets and instructors, a cohort readiness rollup (average, distribution, count not-yet-started). *(Counters shipped.)*
- **Cadet consent screen** — shown once on enrollment; re-summarised in `/settings`.

Reuse the existing account-page chrome and card components; do not hand-roll new ones.

8. Build phases

Phase	Scope	Status	Effort
A	Server-side progress projection (<code>study_progress</code> + <code>studyProgressSync</code>)	done	—
A2	Cohort readiness report + <code>/business/admin</code> for the org owner	done	—
B	Consent — <code>org_seats.consent</code> , the cadet consent/revoke route, blanking in <code>cohortRow()</code>	next — DPIA gate	S
C	<code>org_members</code> + <code>orgStaff()</code> — instructor and admin roles beyond the single owner		M
D	Instructor cohort view + per-cadet drill-down		M
E	Seat management UI + <code>org_assignments</code> reading-path assignment		M

Phase B is small and blocking: the dashboard is live for one owner today, and consent has to exist before a second school's cadets are on it.

9. Open decisions (you)

1. **Logbook visibility** — instructors see study data only (recommended and currently true by construction), or the logbook too? Privacy says study-only. Changing this would mean a new join into `flights` ; don't.
2. **Enrollment** — owner-enters-emails (shipped, more control) vs a join-code (less friction, weaker control)? Could support both.
3. **Org creation** — Fly GACA-on-contract via `grant-org.mjs` (shipped, recommended for a paid B2B product) vs self-serve signup.
4. **Consent default for the pilot school already on the dashboard** — retro-consent (email them, blank readiness until they accept) or grandfather? Recommend retro-consent; it is the defensible answer in a DPIA.
5. **DPIA** — cross-user data sharing must be in the PDPL DPIA scope. This is a launch gate; the instructor dashboard cannot go wide until it's covered.

10. Risks

- **Consent gap.** The live report has no consent check (§3). Highest-priority fix; it is a compliance risk, not just a feature gap.
- **A leaky projection.** The risk moved from "a rules mistake" to "a route returns one column too many". Mitigations: keep the projection in `org-core.ts` where it is pure and unit-tested; assert the exact response shape in the route test; never `SELECT *` into a cohort response.

- **PDPL.** Cross-user visibility is exactly what a DPIA scrutinises — do not go wide before it's covered.
- **Stale summaries.** `studyProgressSync` is upload-only and best-effort: offline, or with no API configured, nothing uploads and the instructor sees old data. Mitigation: the report already carries `progressUpdatedAt` — surface it in the UI so staleness is visible rather than silent.
- **Scope creep.** Resist messaging, grading, attendance, billing. v1 = see the cohort's readiness. That alone sells seats.